CSIS-225 Final Semester Project Report

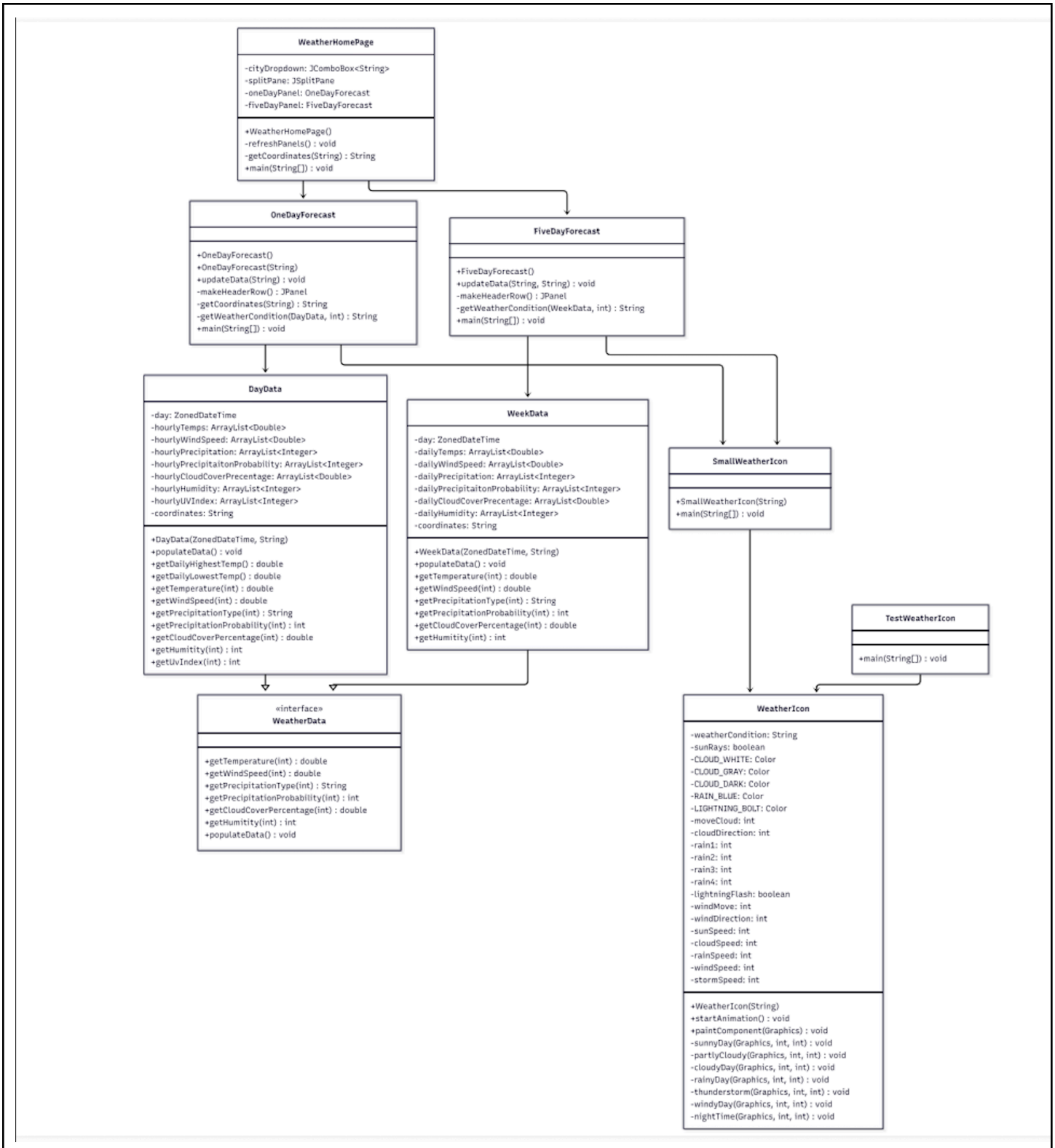
Team Number: 4
Team Name: A PLSS
Team Members: Sanju Arikatla, Sana Seth, Patrick Mcfee, Lucas Davey
Application Title: Weather App

Instructions for Running the Application

Go to the WeatherHomePage.java file in VS Code. In a GitBash terminal, run javac *.java and then java WeatherHomePage. You have 25 tokens to use; once you run out, you will need to create a new account on the tomorrow.io website to get more free tokens. After you receive your API token, go to the DayData.java file and paste your code around line 56. Then do the same for WeekData.java at line 54. If you do not want to create an account, we have also included extra API keys to use in case the tokens run out and you get an error when running the program. They are commented out on lines 57-59 in the DayData.java file, so all you have to do is copy and paste one into line 56. For WeekData.java, they are commented out on lines 55-57 and do the same (copy and paste one into line 54).

Describe how the project reflects a meaningful object-oriented design, making effective use of interfaces, abstract classes, inheritance, etc. (400 - 500 words). If your project has a reasonable number of classes, including a UML diagram here would be helpful, but is not required.

Our project reflects a meaningful object oriented design by organizing the program into multiple classes that each handle a small and specific part for the overall weather application. By breaking up our responsibilities we made the code easier to understand, easier to debug, and run more efficiently. Each class has a clear purpose, which helped us to avoid having one large, complicated file. This also demonstrates encapsulation, since each class keeps its data private and only exposes necessary functionality through public methods. This improves security and prevents unintended changes to important weather data. This allowed different parts of the program to work together in a structured way. We also made an effective use of interfaces, inheritance, and even polymorphism. For example, in this project we used the interface WeatherData which defined behaviors such as retrieving temperature, precipitation, wind speed, and other weather-related values. This interface was important for allowing other classes, such as DayData and WeekData, to implement the real world data from the Weather API that we used. This demonstrates polymorphism because both classes can be used in similar ways, even though they store and process data differently. This allowed us to organize the data in a consistent and reusable way. Essentially, this was a key piece for making the program relevant for the users in their everyday life by being able to access live weather reports. Another concept of object oriented programming that we used was inheritance. We demonstrated inheritance in several classes such as WeatherHomePage, OneDayForecast, FiveDayForecast, and WeatherIcon. These classes extend either JFrame or JPanel which helped us build the graphical user interface using Java Swing. By extending these classes, we were able to reuse built-in functionality like layout management instead of writing everything from scratch. This made the development process faster and more efficient. In addition, our design allowed for each class to focus on one responsibility. For example, the data classes (DayData and WeekData) handle retrieving and storing weather data, while the GUI classes (OneDayForecast and FiveDayForecast) are responsible for displaying that data to the user. The WeatherHomePage class acts as the controller by connecting user input, such as selecting a city, to updates in the display. This separation made it easier to update or fix one part of the program without affecting everything else. Overall, our use of object oriented design allowed us to build a program that is organized, scalable, and easier to maintain. It also made it possible to integrate real-time data and interactive features in a clean and structured way, which improved both the functionality and usability of the application.



List one class that implements event driven programming including Java graphics and Java Swing components.

One class that implements event driven programming is WeatherIcon. This class uses Java Swing and Graphics to draw animated weather icons such as rain, clouds, and sun. It responds to events through repaint(), which updates the drawing whenever the animation changes. The class overrides paintComponent() to draw the shapes and visuals for each weather condition. This demonstrates

event-driven programming because the GUI updates are triggered by events such as repaint() calls instead of running in a fixed sequence. JPanel responds dynamically to these events making the interface interactive and responsive.

List one class that implements threads. Then, describe the purpose and lifecycle of each treaded object (50 - 100 words).

One class that implements threads is WeatherIcon. Its purpose is to create an animated icon for each weather condition instead of a generic image so it looks more visually appealing. The thread works inside of the startAnimation() method as it constantly updates the values of each animation, such as the moving clouds, rain falling at different times, and the sun rays/lightning bolt flashing. We then use repaint() at the end of each loop to update the appearance of the icon. The lifecycle of the thread object lasts until the program ends.

List up to 3 different data structures that have significant uses in the application and the class(es) in which they are used. Then, describe why you chose to use each (50 - 100 words per data structure).

One data structure that was used was an ArrayList inside of the WeekData class. This was used to store the weather data for multiple days such as temperature, wind speed, precipitation, and cloud cover. This was chosen as an ArrayList because it allowed for easy storage and access to multiple values using indexes, which works well when looping through the five day forecast. We chose an ArrayList instead of a standard array because it can resize dynamically and makes it easier to manage and iterate through the data.

Another data structure that was used was a string array inside of the WeatherHomePage file. We used this to store the different cities for the dropdown menu on the main home page. It was best to use an array to store the cities because it allows the JComboBox to have a constructor that accepts a string array. There is also no need for any dynamic sizing since there are only the set options of cities.

Another data structure that was used was an ArrayList inside of the DayData class. This was used to store hourly weather data such as temperature, precipitation, and wind speed. Each index in the ArrayList represents a specific hour, which makes it easy to access and display data for all 24 hours in the one day forecast. This structure was useful because it keeps the data organized and allows for efficient looping when building the hourly table in the GUI.This structure also makes the code easier to read and work with, since each index matches a specific hour, which helps make the data more clear and easier to debug.

Describe the Java or general programming feature or construct included in your project that was not specifically studied in this class or prerequisite classes. Include the class(es) in which this feature is implemented. (200 - 300 words)

The java programming concept used that was not mentioned in the course was initiating API calls to retrieve real time weather data. The files that implement this concept are the WeekData.java and the DayData.java files in the repository. The API that was used to accomplish this was tomorrow.io . To initiate the API call, a URL that contains all proper fields and valid API key is constructed. The next step is to use the java HttpUrlConnection class to open a http connection so the data retrieval process can begin. Once http response code 200 (code for OK) is received, data output in JSON format is read into a stringbuilder. Once the data is transferred into the string builder, the HTTP connection is closed and the process to separate the data begins. One challenge that was encountered was handling errors when too many API requests were made. This showed the importance of managing how often requests are sent and designing the program efficiently. Overall, using API calls allowed our application to display live and accurate weather information instead of hardcoded values, which made the program more realistic and useful.

Describe how your program focuses on social good to create positive societal, environmental, or community impact (100 - 300 words).

Our program focuses on social good by helping users make safe daily decisions based on weather conditions. One specific feature we used to focus on social good was one that displays the UV index and it provides a quick recommendation message depending on the level. For example if the UV is above 5, then a message will print that warns the users and tells them to avoid sunlight and time spent outdoors. This feature will make people more aware of the risks that come along with being exposed to a high UV for an extended period of time. It also presents it in an easy and simple to understand way. Another feature that was included was a daily quote each time the program runs to ‘brighten’ up someone’s day each time they check the weather. Additionally there are more safety tips included based on weather conditions. For example, if the weather is above 90 degrees, there will be a message saying “Hot weather: Drink water and avoid going outside too long.” This allows people to remain aware of how weather can impact them. This can be especially helpful for people who may not always check detailed forecasts, since it gives them quick and important information right away. It also helps users build better daily habits by reminding them to stay safe in different weather conditions. Overall, it makes the application not just informative, but also practical and helpful in everyday life.